Linked List

N S Kumar
Professor
Department of Computer Science &
Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: ADT and List

An Abstract Data type specifies

a) Both interface and implementation

b) Only Implementation

c) Only interface

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: ADT and List

An Abstract Data type specifies

a) Both interface and implementation

b) Only Implementation

c) Only interface

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: ADT and List

An array is a data structure in which

a) Access is faster and insertion/deletion are not

b) Both access and insertion/deletion are fast

c) Both access and insertion/deletion are slow

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: ADT and List

An array is a data structure in which

a) Access is faster and insertion/deletion are not

b) Both access and insertion/deletion are fast

c) Both access and insertion/deletion are slow

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: ADT and List

A list is a data structure in which

a) Access is faster and insertion/deletion are not

b) Both access and insertion/deletion are fast

c) Both access and insertion/deletion are slow

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: ADT and List

A list is a data structure in which

a) Access is faster and insertion/deletion are not

b) Both access and insertion/deletion are fast

c) Both access and insertion/deletion are slow

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: ADT and List

What is true of a list?

a) Can access the ith element directly

b) Cannot be empty

c) Every element has a position

d) Can grow indefinitely

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: ADT and List

What is true of a list?

a) Can access the ith element directly

b) Cannot be empty

c) Every element has a position

d) Can grow indefinitely

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: ADT and List

Which of the following on a list always requires less operations

a) Add in the beginning

b) Add in the middle

c) Find the midpoint

d) Remove an element at the end

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: ADT and List

Which of the following on a list always requires less operations

a) Add in the beginning

b) Add in the middle

c) Find the midpoint

d) Remove an element at the end

© Confidential Information of PES University. All Rights Reserved - 2026

Evolving a Linked List

- Self referential data structure :
  - Is a structure where a field refers to the same structure type
  - Is implemented as a pointer to the same structure

```
struct node
{
    int key_;
    struct node *next_;
};
typedef struct node node_t;
```


© Confidential Information of PES University. All Rights Reserved - 2026

Evolving a Linked List

What happens here?

```
node_t *p;
p = (node_t*)malloc(sizeof(node_t));
p->key_ = 20;
```


© Confidential Information of PES University. All Rights Reserved - 2026

Evolving a Linked List

How about now?

```
node_t *p;
p = (node_t*)malloc(sizeof(node_t));
p->key_ = 20;

p->next_ = (node_t*)malloc(sizeof(node_t));
p->next_->key_ = 40;
```


© Confidential Information of PES University. All Rights Reserved - 2026

Evolving a Linked List

Some more code?

```
node_t *p;
p = (node_t*)malloc(sizeof(node_t));
p->key_ = 20;

p->next_ =
(node_t*)malloc(sizeof(node_t));
p->next_->key_ = 40;

p->next_->next_ =
(node_t*)malloc(sizeof(node_t));
p->next_->next_->key_ = 60;
p->next_->next_->next_ = NULL;
```


© Confidential Information of PES University. All Rights Reserved - 2026

Evolving a Linked List

```
void disp(node_t* q)
{
    while(q != NULL)
    {
        printf("%d ", q->key_);
        q = q->next_;
    }
}

disp(p);
```

What is this code? q = q->next_;

Why not q++?

So, We have engineered a linked list!

© Confidential Information of PES University. All Rights Reserved - 2026

A list is either empty or has a node followed by a list

The nodes are accessed starting from the head of the list

The access is sequential.

In the worst case, we may have to traverse all the nodes before reaching the last node. This may require n operations if there are n nodes.

An ordered list is a list where the nodes are arranged in the order of keys.

Nodes should be inserted based on the comparison of the keys.

Nodes can be inserted in an empty list or in the beginning or the middle or the end.

© Confidential Information of PES University. All Rights Reserved - 2026

Structure of list :

- The structure of node is similar to what we discussed before.
  - A key field and a pointer field to provide for self reference

```
struct node
{
    int key_;
    struct node *next_;
};
typedef struct node node_t;
```

- List has a head which will point to the first node in the list.

```
struct mylist
{
    node_t *head_;
};

typedef struct mylist mylist_t;
```


© Confidential Information of PES University. All Rights Reserved - 2026

Ordered List

- Interface : Similar to the last example

- We require functions for creating and destroying the list.
- We require functions to insert into the list in the right position.
- We will also provide a way of traversing and displaying the list.

- void init_list(mylist_t*);
- void insert_list(mylist_t*, int key);
- void disp_list(mylist_t*);
- void free_list(mylist_t*);

© Confidential Information of PES University. All Rights Reserved - 2026

Ordered List

- Initially list is empty. The head_ of the list is grounded.

```
void init_list(mylist_t* ptr_list)
{
    ptr_list->head_ = NULL;
}
```

- This function displays all the keys in the list starting from the head_.

```
void disp_(node_t* q)
{
    while(q != NULL)
    {
        printf("%d ", q->key_);
        q = q->next_;
    }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Ordered List

- Clear the list
  - Observe the code in the while.
  - Can we change the order of these statements?

```
void free_list_(node_t* q)
{
    node_t* r;
    while(q != NULL)
    {
        r = q->next_;
        free(q);
        q = r;
    }
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Ordered List

- Insertion :
- Make a new node; populate it; insert
- Boundary conditions :
  - Empty list
  - Add in the beginning
  - Add in the middle
  - Add at the end

```
// 1.empty list :
// head_ is NULL in the beginning
if(ptr_list->head_ == NULL)
{
    ptr_list->head_ = temp;
    temp->next_ = NULL;
}
```


© Confidential Information of PES University. All Rights Reserved - 2026

Ordered List

- Insertion :
  - Add in the beginning
  - Add in the middle
  - Add at the end
- Traverse the list

```
node_t* pres = ptr_list->head_;
while(pres != NULL && pres->key_ < temp->key_ )
{
    pres = pres->next_;
}
```

- Cannot traverse back
- Solution :
  - One trailing pointer or
  - Look ahead

© Confidential Information of PES University. All Rights Reserved - 2026

Ordered List

- Insertion :
- Traverse the list : use a trailing pointer

```
node_t* prev = NULL;
node_t* pres = ptr_list->head_;
while(pres != NULL && pres->key_ < temp->key_ )
{
  prev = pres;
  pres = pres->next_;
}
```

- prev tags behind the pres
- temp is to be inserted between the nodes to which prev and pres point to.
- What if prev is NULL => insert in the beginning
- Check : insert in the middle or end will have similar code

© Confidential Information of PES University. All Rights Reserved - 2026

Ordered List

- Insertion :

```
if(prev == NULL) // insert in the beginning
{
  temp->next_ = pres;
  ptr_list->head_ = temp;
}
else // middle or end
{
  temp->next_ = pres;
  prev->next_ = temp;
}
```

We have successfully engineered an ordered list!!

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor
Department of Computer Science & Engineering (AI & ML)